MLP Coursework 2

S2816575

Abstract

In this report we study the problem of overfitting, which is the training regime where performance increases on the training set but decrease on validation data. Overfitting prevents our trained model from generalizing well to unseen data. We first analyse the given example and discuss the probable causes of the underlying problem. Then we investigate how the depth and width of a neural network can affect overfitting in a feedforward architecture and observe that increasing width and depth tend to enable further overfitting. Next we discuss how two standard methods, Dropout and Weight Penalty, can mitigate overfitting, then describe their implementation and use them in our experiments to reduce the overfitting on the EMNIST dataset. Based on our results, we ultimately find that both dropout and weight penalty are able to mitigate overfitting.

We further explore alternative loss and activation functions, and evaluate whether their performance relates to problems of overfitting.

Finally, we conclude the report with our observations and related work. Our main findings indicate that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

1. Introduction

In this report we focus on a common and important problem while training machine learning models known as overfitting, or overtraining [1], which is the training regime where performances increase on the training set but decrease on unseen data. We first start with analysing the given problem in Figure 1, study it in different architectures and then investigate different strategies to mitigate the problem. In particular, Section 2 identifies and discusses the given problem, and investigates the effect of network width and depth in terms of generalization gap (see Ch. 5 in Goodfellow et al. 2016) and generalization performance. Section 3 introduces two regularization techniques to alleviate overfitting: Dropout (Srivastava et al., 2014) and L1/L2 Weight Penalties (see Section 7.1 in Goodfellow et al. 2016). We first explain them in detail and discuss why they are used for alleviating overfitting. We extend our study to implement a combined L1 and L2 penalty.

In Section 4, we incorporate each of them and their various combinations to a three hidden layer[2] neural network, train it on the EMNIST dataset, which contains 131,600 images of characters and digits, each of size 28x28, from 47 classes. We evaluate them in terms of generalization gap and performance, and discuss the results and effectiveness of the tested regularization strategies. Our results show that both dropout and weight penalty are able to mitigate overfitting. We end Section 4, by testing an alternative activation function, and explore whether the observed performance degradation relates to overfitting or some other problem.

Finally, we conclude our study in Section 5, noting that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

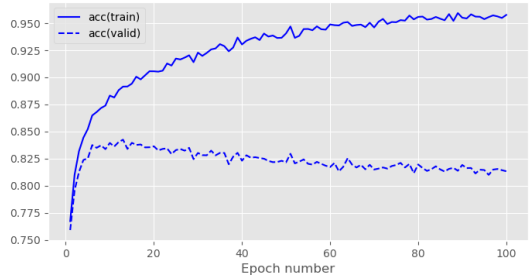
2. Problem identification

Overfitting to training data is a very common and important issue that needs to be dealt with when training neural networks or other machine learning models in general (see Ch. 5 in Goodfellow et al. 2016). A model is said to be overfitting when as the training progresses, its performance on the training data keeps improving, while its is degrading on validation data. Effectively, the model stops learning related patterns for the task and instead starts to memorize specificities of each training sample that are irrelevant to new samples. Overfitting leads to bad generalization performance in unseen data, as performance on validation data is indicative of performance on test data and (to an extent) during deployment.

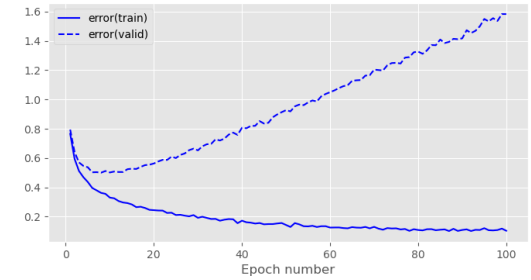
Although it eventually happens to all gradient-based training, it is most often caused by models that are too large with respect to the amount and diversity of training data. The more free parameters the model has, the easier it will be to memorize complex data patterns that only apply to a restricted amount of samples. A prominent symptom of overfitting is the generalization gap, defined as the difference between the validation and training error. A steady increase in this quantity is usually interpreted as the model

[1] Technically, overtraining is the act of training beyond a point at which performance degrades; while overfitting is training to the extent that the learnt function is more complex than required to capture the training data. They correlate, but they are not, strictly speaking, the same thing. To keep things simple, and according to common usage of the terms, we are using them interchangeably in this document, and referring to their joint effect.

[2] We denote all layers as hidden except the final (output) one. This means that depth of a network is equal to the number of its hidden layers + 1.



(a) accuracy by epoch



(b) error by epoch

Figure 1. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for the baseline model.

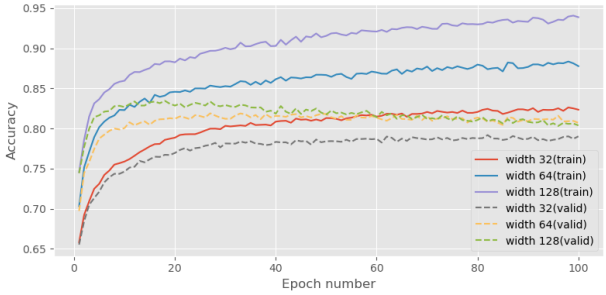
entering the overfitting regime.

Figure 1a and 1b show a prototypical example of overfitting. We see in Figure 1a that **[the *training accuracy* increases steadily throughout training, eventually approaching a high value close to saturation** (~ 0.950)**. In contrast, the *validation accuracy* initially increases but then plateaus and starts decreasing around epoch** $\sim 5-10$**, after which it fluctuates slightly without any further systematic improvement. At the final epoch, the validation accuracy is around** ~ 0.810**, which is a significant decrease. Figure 1b presents the corresponding cross-entropy errors. The *training error* decreases monotonically, while the *validation error* initially falls but then stops decreasing and begins to level off around the same epoch range** ($5-10$)**. From that point onward, the training error continues to decrease whereas the validation error begins to steadily increase. This divergence between the training and validation curves indicates an increasing *generalisation gap*, which is the difference between the validation error and training error. The generalization gap at the final epoch is around** ~ 1.5**. The model fits the training data more and more closely but no longer improves. This is a clear indication of overfitting, as the model is learning the training data too well and is not generalising to the validation data.]** .

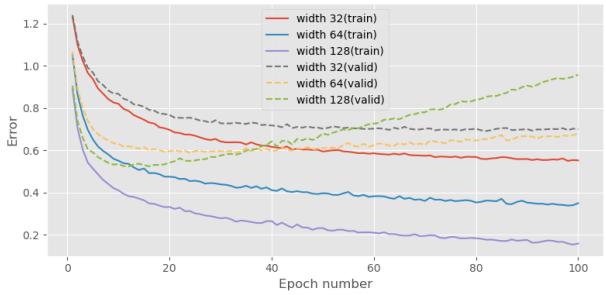
The extent to which our model overfits depends on many factors. For example, the quality and quantity of the training set and the complexity of the model. If we have sufficiently many diverse training samples, or if our model contains few hidden units, it will in general be less prone to overfitting.

# Hidden Units	Val. Acc.	Train Error	Val. Error
32	79.0%	0.552	0.700
64	80.7%	0.350	0.678
128	80.2%	0.156	0.973

Table 1. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network widths on the EMNIST dataset.



(a) accuracy by epoch



(b) error by epoch

Figure 2. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network widths.

Any form of regularization will also limit the extent to which the model overfits.

2.1. Network width

First we investigate the effect of increasing the number of hidden units in a single hidden layer network when training on the EMNIST dataset. The network is trained using the Adam optimizer with a learning rate of 10^{-3} and a batch size of 100, for a total of 100 epochs.

The input layer is of size 784, and output layer consists of 47 units. Three different models were trained, with a single hidden layer of 32, 64 and 128 ReLU hidden units respectively. Figure 2 depicts the error and accuracy curves over 100 epochs for the model with varying number of hidden units. Table 1 reports the final accuracy, training error, and validation error. We observe that **[varying the number of hidden units has a clear impact on both performance and overfitting. Table 1 and Figure 2a show that the network with** 32 **hidden units achieves the lowest validation accuracy** (72%) **and the highest validation error** (0.700)**, while the models with** 64 **and**

128 hidden units perform better on the validation set. In particular, the 128-unit model attains the smallest training error at the end of training, 0.156. However, Figure 2b also reveals that the 128-unit model exhibits a larger gap between training and validation error: its training error continues to decrease while the validation error flattens and rapidly increases after around epoch ~ 5. This indicates stronger overfitting compared to the 64-unit model, whose training and validation errors remain closer together, though the validation error does increase after epoch ~ 60, albeit at a slower rate, indicating overfitting in that case as well. The 32-unit model shows little signs of overfitting, having a generalization gap of around ~ 0.1 at the final epoch. Thus, while increasing width from 32 to 128 hidden units improves validation accuracy, it also amplifies the generalisation gap, with the widest model overfitting the most.] .

[Our width experiment confirms the expected trade-off between model capacity, performance, and overfitting. As we increase the number of hidden units from 32 to 64 and 128, training error decreases monotonically and training accuracy increases (or stays around the same in the case of the 128- and 64- unit models), showing that wider networks fit the training data more easily. Validation performance initially benefits from this increase in capacity: moving from 32 to 64 hidden units improves validation accuracy and reduces validation error. However, beyond a certain point, additional capacity primarily reduces training error without translating into better validation performance. In our results, the 128-unit model achieves the lowest training error but exhibits a larger generalisation gap of around ~ 0.8. This is significantly larger than the 64-unit model(gap around ~ 0.4). Consequently, the 128-unit model overfits more than the 64-unit model, even though it has the best training performance. The 64-unit model provides a better balance between training fit and generalisation, overfitting less while still outperforming the 32-unit model on the validation set. This behaviour aligns with standard bias-variance and capacity arguments in the neural network literature. Increasing capacity reduces bias [3] but increases variance [4] and the risk of overfitting if regularisation is insufficient (e.g. (Goodfellow et al., 2016), chapter 5.4.4). Our empirical findings match these theoretical expectations moderate width improves generalisation, whereas excessive width mainly amplifies overfitting.] .

2.2. Network depth

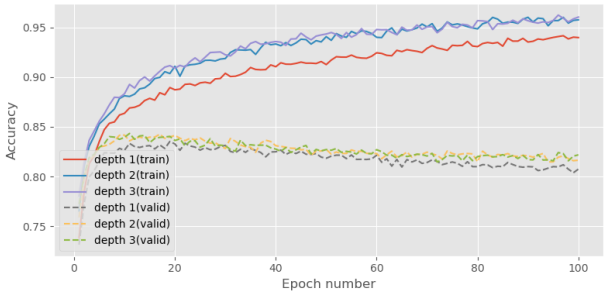
Next we investigate the effect of varying the number of hidden layers in the network. Table 2 and Figure 3 depict

[3]Here, bias can be defined as $\text{bias}(\hat{\theta}_m) = \mathbb{E}[\hat{\theta}_m] - \theta_m$ where $\hat{\theta}_m$ is the expecation over the dataset and θ_m is the true underlying function to be learned.

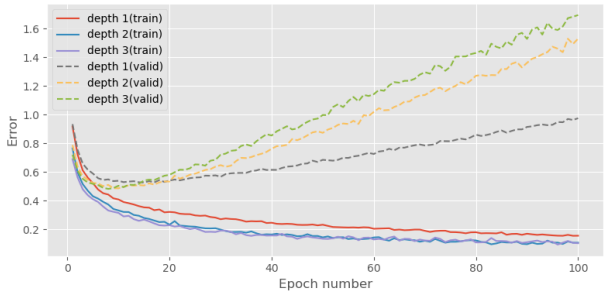
[4]$Var[\hat{\theta}]$, where $\hat{\theta}$ is the estimator. The variance of an estimator provides a measure of how we would expect the estimate we compute from data to vary as we independently resample the dataset from the underlying data-generating process.

# Hidden Layers	Val. Acc.	Train Error	Val. Error
1	0.807	0.156	0.975
2	0.816	0.107	1.529
3	0.822	0.104	1.693

Table 2. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network depths on the EMNIST dataset.



(a) accuracy by epoch



(b) error by epoch

Figure 3. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network depths.

results from training three models with one, two and three hidden layers respectively, each with 128 ReLU hidden units. As with previous experiments, they are trained with the Adam optimizer with a learning rate of 10^{-3} and a batch size of 100.

We observe that [varying the number of hidden layers also affects both performance and overfitting. Table ?? and Figure 3a show that a network with only 1 hidden layer underperforms: it attains the lowest validation accuracy and the highest validation error, and both training and validation errors remain relatively high, indicating underfitting.

Adding more depth improves validation performance. The 2- and 3-layer networks achieve substantially higher validation accuracy and lower validation error than the 1-layer model. However, Figure ?? reveals that the deeper networks also display larger gaps between training and validation error. For the 3-layer model in particular, training error continues to decrease while validation error plateaus or slightly increases after around epoch [X], indicating stronger overfitting. The 2-layer

model offers a better compromise, showing improved validation performance over the 1-layer baseline while maintaining a somewhat smaller generalisation gap than the 3-layer model. Overall, depth improves representational power but also increases the tendency to overfit.] .

[Our findings indicate that varying the number of hidden layers affects the results in a consistent manner. As we increase the depth from 1 to 2 hidden layers, we observe a significant improvement in validation accuracy from 68.3% to 81.0%, along with a corresponding decrease in both training and validation error. This trend suggests that adding more layers allows the model to capture more complex patterns in the data, leading to better generalization. However, when we further increase the depth to 3 hidden layers, we notice a slight increase in validation accuracy to 81.7%, but also a substantial increase in validation error to 1.209, indicating overfitting. These results align with established theories in neural network literature, which suggest that while increasing depth can enhance model capacity and performance, it also raises the risk of overfitting if not properly regularized. Overall, our expectations based on prior knowledge are confirmed by our experimental results] .

3. Regularization

In this section, we investigate three regularization methods to alleviate the overfitting problem, specifically dropout layers, the L1 and L2 weight penalties and label smoothing.

3.1. Dropout

Dropout (Srivastava et al., 2014) is a stochastic method that randomly inactivates neurons in a neural network according to an hyperparameter, the inclusion rate (*i.e.* the rate that an unit is included). Dropout is commonly represented by an additional layer inserted between the linear layer and activation function. Its forward pass during training is defined as follows:

$$\text{mask} \sim \text{bernoulli}(p) \tag{1}$$

$$\boldsymbol{y}' = \text{mask} \odot \boldsymbol{y} \tag{2}$$

where $\boldsymbol{y}, \boldsymbol{y}' \in \mathbb{R}^d$ are the output of the linear layer before and after applying dropout, respectively. mask $\in \mathbb{R}^d$ is a mask vector randomly sampled from the Bernoulli distribution with inclusion probability p, and $\odot$ denotes the element-wise multiplication.

At inference time, stochasticity is not desired, so no neurons are dropped. To account for the change in expectations of the output values, we scale them down by the inclusion probability p:

$$\boldsymbol{y}' = \boldsymbol{y} * p \tag{3}$$

As there is no nonlinear calculation involved, the backward propagation is just the element-wise product of the gradients with respect to the layer outputs and mask created in the forward calculation. The backward propagation for dropout is therefore formulated as follows:

$$\frac{\partial \boldsymbol{y}'}{\partial \boldsymbol{y}} = mask \tag{4}$$

Dropout is an easy to implement and highly scalable method. It can be implemented as a layer-based calculation unit, and be placed on any layer of the neural network at will. Dropout can reduce the dependence of hidden units between layers so that the neurons of the next layer will not rely on only few features from of the previous layer. Instead, it forces the network to extract diverse features and evenly distribute information among all features. By randomly dropping some neurons in training, dropout makes use of a subset of the whole architecture, so it can also be viewed as bagging different sub networks and averaging their outputs.

3.2. Weight penalty

L1 and L2 regularization (Ng, 2004) are simple but effective methods to mitigate overfitting to training data. The application of L1 and L2 regularization strategies could be formulated as adding penalty terms with L1 and L2 norm square of weights in the cost function without changing other formulations. The idea behind this regularization method is to penalize the weights by adding a term to the cost function, and explicitly constrain the magnitude of the weights with either the L1 and L2 norms. The optimization problem takes a different form:

$$\text{L1: } \min_{\boldsymbol{w}} E_{\text{data}}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w}) + \lambda \|w\|_1 \tag{5}$$

$$\text{L2: } \min_{\boldsymbol{w}} E_{\text{data}}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w}) + \lambda \|w\|_2^2 \tag{6}$$

where E_{data} denotes the cross entropy error function, and $\{\boldsymbol{X}, \boldsymbol{y}\}$ denotes the input and target training pairs. λ controls the strength of regularization.

Weight penalty works by constraining the scale of parameters and preventing them to grow too much, avoiding overly sensitive behaviour on unseen data. While L1 and L2 regularization are similar to each other in calculation, they have different effects. Gradient magnitude in L1 regularization does not depend on the weight value and tends to bring small weights to 0, which can be used as a form of feature selection, whereas L2 regularization tends to shrink the weights to a smaller scale uniformly.

3.2.1. Combination of L1 and L2 Regularisation

[We consider a combined regularisation approach that incorporates both L1 and L2 penalties to leverage the strengths of each. A naive combination simply adds both penalties to the loss:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \left(\|\mathbf{w}\|_1 + \|\mathbf{w}\|_2^2 \right), \tag{7}$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss, w denotes the vector of trainable weights, and $\lambda > 0$ controls the overall

strength of regularisation. This formulation, however, does not allow us to independently control the contributions of L1 (sparsity) and L2 (weight decay). The gradient contributions from both penalties could interfere with each other, potentially leading to suboptimal weight updates. Using Adam's Learning rule, the gradient update for weight w_i at time step t would follow the following steps:

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1)\nabla_{w_i}\mathcal{L} \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2)(\nabla_{w_i}\mathcal{L})^2 \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t} \\
\hat{v}_t &= \frac{v_t}{1-\beta_2^t} \\
w_i &= w_i - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}
\end{aligned}
$$

where η is the learning rate, β_1 and β_2 are the exponential decay rates for the moment estimates, and ϵ is a small constant to prevent division by zero. We have:

$$
\nabla_{w_i}\mathcal{L} = \nabla_{w_i}\mathcal{L}_{CE} + \underbrace{\lambda \cdot \mathbf{sign}(w_i)}_{\text{L1 component}} + \underbrace{2\lambda w_i}_{\text{L2 component}}
$$

Thus, we observe that the L2 regularisation term contributes a gradient proportional to w_i, i.e., $\nabla_{w_i}(\|\mathbf{w}\|_2^2) = 2w_i$, whereas the L1 term contributes a constant magnitude $|\nabla_{w_i}(\|\mathbf{w}\|_1)| = 1$ regardless of weight magnitude. Consequently, for $|w_i| \ll 1$, the ratio $\frac{|2\lambda w_i|}{\lambda} = 2|w_i|$ is small, meaning the L1 component dominates and drives weights toward zero (promoting sparsity). Conversely, for $|w_i| \gg 1$, the L2 gradient $2\lambda w_i$ grows linearly with $|w_i|$ and exerts a stronger shrinkage effect. However, typical weight initialization schemes (e.g., (**Glorot & Bengio, 2010**), whose random uniform weight initalizer is utilized accross our experiments) set $|w_i| \sim O(10^{-2})$ [5], and the L2 penalty continuously discourages large weights. As a result, weights may remain in a regime where $|w_i|$ is small, limiting the relative influence of the L2 term and potentially reducing its regularisation benefits during training. To address this, we use a weighted combination reminiscent of elastic net regularisation (**Zou & Hastie, 2005**):

$$
\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda\left(\alpha\|\mathbf{w}\|_1 + (1-\alpha)\frac{1}{2}\|\mathbf{w}\|_2^2\right), \tag{8}
$$

where $0 \le \alpha \le 1$ controls the trade-off between L1 and L2 regularisation. For $\alpha \approx 1$ we emphasise sparsity, resulting in almost pure L1 reguralization, whereas smaller α places more weight on L2, which shrinks weights smoothly and tends to improve stability and generalisation. Thus, we have a new hyperparameter α to tune the balance between the two regularisation effects, allowing us to harness the complementary benefits of both methods. This approach provides a more flexible and effective way to regularise neural networks, potentially leading to improved generalisation performance. One might find this combined approach particularly beneficial in scenarios where both sparsity and weight decay are desired, such as when dealing with high-dimensional data or when aiming to reduce model complexity while maintaining predictive accuracy (since the L2 term will prevent overfitting my keeping all weights relatively small). We note that one could also consider simply defining separate hyperparameters for L1 and L2 regularisation strengths (i.e., λ_1 and λ_2), but this would increase the hyperparameter search space and complexity. The elastic net formulation with a single λ and balance parameter α provides a more tractable approach while still allowing flexible control over the regularisation behaviour.] .

[5] The authors derive the Xavier variance, $\text{Var}[w] = \frac{2}{n_{in}+n_{out}}$, where n_{in} and n_{out} are the number of input and output units, respectively. For typical layer sizes (e.g., $n_{in}, n_{out} \sim 100$), this yields $\text{Var}[w] \sim 10^{-2}$, so weights are initialized with standard deviation ~ 0.1.

3.3. Label smoothing

Label smoothing regularizes a model based on a softmax with K output values by replacing the hard target 0 labels and 1 labels with $\frac{\alpha}{K-1}$ and $1-\alpha$ respectively. α is typically set to a small number such as 0.1.

$$
\begin{cases} \frac{\alpha}{K-1}, & \text{if} \quad t_k = 0 \\ 1-\alpha, & \text{if} \quad t_k = 1 \end{cases} \tag{9}
$$

The standard cross-entropy error is typically used with these *soft* targets to train the neural network. Hence, implementing label smoothing requires only modifying the targets of training set. This strategy may prevent a neural network to obtain very large weights by discouraging very high output values.

4. Balanced EMNIST Experiments

Here we evaluate the effectiveness of the given regularization methods for reducing the overfitting on the EMNIST dataset. We build a baseline architecture with three hidden layers, each with 128 neurons, which suffers from overfitting as shown in section 2.

Here we train the network with a lower learning rate of 10^{-4}, as the previous runs were overfitting after only a handful of epochs. Results for the new baseline (c.f. Table 3) confirm that lower learning rate helps, so all further experiments are run using it.

Here, we apply the L1 or L2 regularization with dropout to our baseline and search for good hyperparameters on the validation set. Then, we apply the label smoothing with $\alpha = 0.1$ to our baseline. We summarize all the experimental results in Table 3. For each method except the label smoothing, we plot the relationship between generalization gap and validation accuracy in Figure 4.

First we analyze three methods separately, train each over a set of hyperparameters and compare their best performing results.

[Given a budget of exactly four experiments to evaluate

Model	Hyperparameter value(s)	Validation accuracy	Train Error	Validation Error
Baseline	-	0.837	0.241	0.533
Dropout	0.6	80.7	0.549	0.593
	0.7	0.841	0.348	0.464
	0.85	***85.1***	***0.329***	***0.434***
	0.97	85.4	0.244	0.457
L1 penalty	*5e-4*	*79.5*	*0.642*	*0.658*
	1e-3	0.733	0.883	0.894
	5e-3	2.41	3.850	3.850
	5e-2	2.20	3.850	3.850
L2 penalty	5e-4	85.1	0.306	0.460
	1e-3	*0.849*	*0.356*	*0.453*
	5e-3	81.3	0.586	0.607
	5e-2	39.2	2.258	2.256
Label smoothing	0.1	0.853	1.020	0.557

Table 3. Results of all hyperparameter search experiments. *italics* indicate the best results per series (Dropout, L1 Regularisation, L2 Regularisation, Label smoothing) and **bold** indicates the best overall.

Model	Hyperparameters λ	α	Validation accuracy	Train Error	Validation Error
Baseline	–	–	0.837	0.241	0.533
L1 and L2 penalty mixed	$\mathbf{5 \times 10^{-4}}$	**0.25**	**0.843**	**0.437**	**0.477**
	5×10^{-4}	0.50	0.829	0.505	0.532
	5×10^{-4}	0.75	0.813	0.587	0.610
	1×10^{-3}	0.25	0.821	0.538	0.563

Table 4. Results of all hyperparameter search experiments for combined L1 and L2 regularisation. **bold** indicates the best results for this experiment. λ is our hyperparameter controlling the overall strength of regularisation, while α is the mixing coefficient, which controls the balance between L1 and L2 penalties.

the proposed L1+L2 regularisation scheme, described in Equation 8, we focus on the following question:

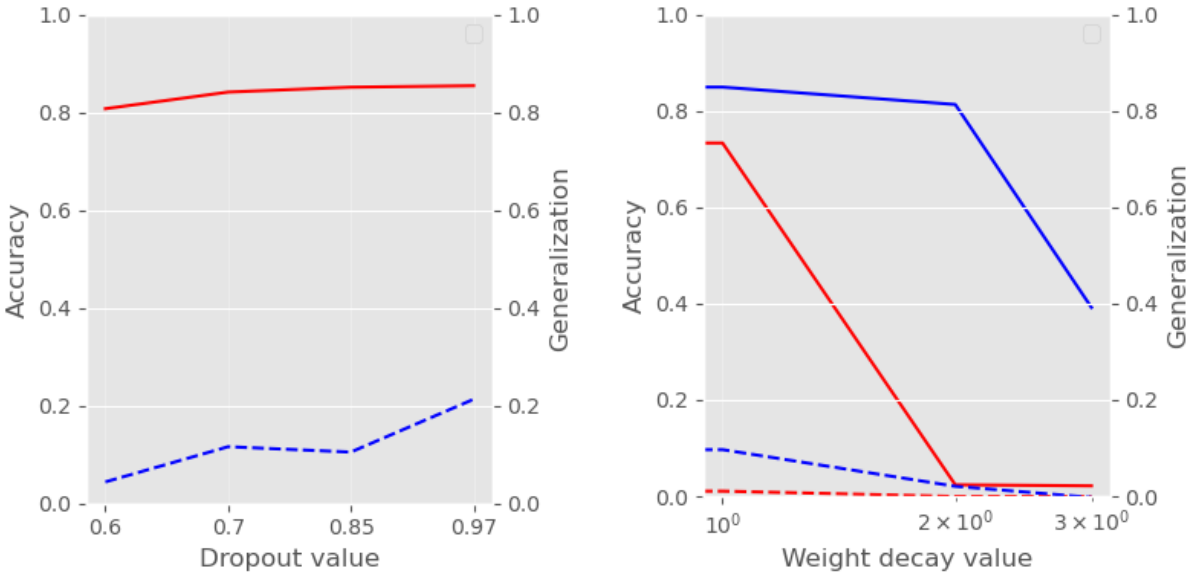
How does the trade-off between L1-induced sparsity and L2-induced weight decay affect the generalisation behaviour of the network, beyond simply optimising validation accuracy?

This question goes beyond hyperparameter tuning: we are interested in understanding *how* different mixtures of L1 and L2 change both performance and the generalisation gap, informed by our earlier experiments and the literature on elastic-net style regularisation. Furthermore, we are interesting in studing the *relationship between α and λ*, by seeing how smaller values for one hyperparameter value might work better larger values of the other one.

Based on the individual L1 and L2 results in Table 3, we select four configurations that span a meaningful range of behaviours:

1. **$\lambda = 5 \times 10^{-4}, \alpha = 0.25$: L2-dominant. This tests whether strong weight decay with limited sparsity can reduce overfitting while preserving stable optimisation.**
2. **$\lambda = 5 \times 10^{-4}, \alpha = 0.50$: Balanced mix. This configuration treats L1 and L2 equally (to some sense) and is expected to encourage both moderate sparsity and smooth shrinkage, potentially giving a good generalisation trade-off.**
3. **$\lambda = 5 \times 10^{-4}, \alpha = 0.75$: L1-dominant. Here we investigate whether emphasising sparsity improves robustness or instead harms performance by discarding useful features.**
4. **$\lambda = 1 \times 10^{-3}, \alpha = 0.25$: Stronger overall regularisation with L2 bias. This examines whether increasing λ primarily reduces overfitting or pushes the model into underfitting. This configuration also allows us to see if a higher λ can compensate for a lower α.**

These four runs, together with our baseline and single-penalty experiments, let us compare how different mixtures of sparsity and weight decay affect both validation accuracy and the generalisation gap. This directly addresses our question and fits with established analyses



(a) Accuracy and error gap with varying dropout values.

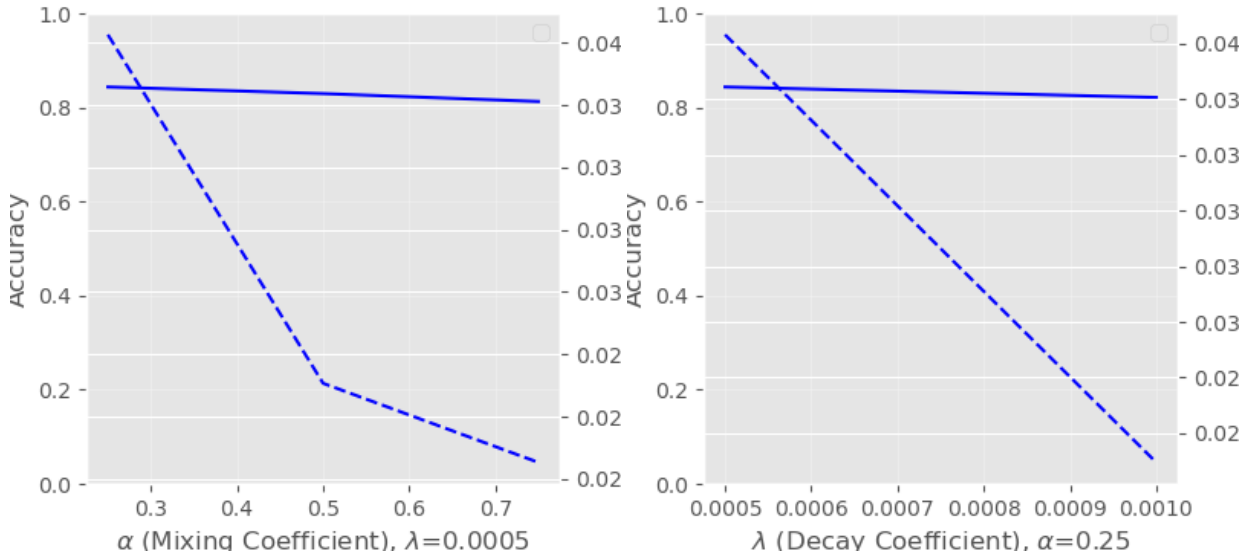
(b) Accuracy and error gap by weight penalty.

Figure 4. Accuracy and error by regularisation strength of each method (Dropout and L1/L2 Regularisation).

of elastic-net regularisation, where the L1/L2 balance is known to shape sparsity patterns and predictive performance.] .

[For the baseline model we use a fully connected network as described at the start of Section 4, trained with cross-entropy loss and hard labels. The soft-labelled baseline replaces the hard labels with smoothed targets, assigning $(1-\epsilon)$ to the true class and distributing ϵ uniformly over the remaining classes with smoothing parameter $\epsilon = 0.1$. Table 3 summarises the validation accuracy and training/validation errors for the baseline, the soft-labelled baseline, dropout, L1- and L2-reguralization experiments with varying hyperparameter values. Table 4 summarises the validation accuracy and training/validation errors for the L1-L2 combination reguralization described in Equation 8. The soft-labelled baseline slightly improves validation accuracy over the hard-label baseline. More specifically, the soft-labelled baseline raised the validation accuracy by $\sim 15\%$. Typically, label-smoothing reduces the generalisation gap, indicating that it can acts as a mild regulariser by discouraging overly confident predictions. However, in out experiments, the training error for the soft-labelled baseline is significantly higher (1.020) than that of the hard-label baseline (0.241), suggesting that the model struggles to fit the smoothed targets. This may be due to the relatively high smoothing parameter $\epsilon = 0.1$, which could be making the targets too ambiguous for effective learning. Among the regularised models, we observe that appropriately tuned dropout and L2 weight decay further improve validation performance and reduce overfitting. For example, a dropout rates of $0.7, 0.85, 0.97$ achieves higher validation accuracy than the baseline as well as a lower validation error $(0.464, 0.434$ and 0.457 respectively vs. 0.533 for the baseline model), keeping the generalization gap smaller than the baseline, suggesting a good bias-variance trade-off. Similarly, the best L2 configuration, with the strength of reguralization $\lambda = 10^{-3}$, reduces the validation error and narrows the gap between training and validation error, indicating improved generalisation. The L2 experiment results were similar for $\lambda = 5 \times 10^{-4}$ and $\lambda = 10^{-3}$ in terms of accuracy, with a difference of 0.2%. However, in the latter case, we have a slighly lower validation error of 0.453. Thus, we deduce that a very small decrease in accuracy vs. a slighly lower validation error is a reasonable tradeoff in this case. Making the optimal hyperparameter choice for L2 reguralization in our case $\lambda = 10^{-3}$. Our experiment results show that L1 reguralization does not outperform L2 reguralization in our experiments, with the best L1 configuration $(\lambda = 5 \times 10^{-4})$ achieving lower validation accuracy and higher validation error than the best L2 model, as well as the baseline. This suggests that weight decay is more effective than sparsity-inducing regularisation for this task and architecture.

Table 4 depicts our experiment results for the combined L1 and L2 reguralisation model. None of our expirments outperform the Dropout-reguralization model. The best configuration $(\lambda = 5 \times 10^{-4}, \alpha = 0.25)$ achieves a validation accuracy of 84.3% and a validation error of 0.477, which is better than the best L1 model but still falls short of the best L2 and dropout models. However we do note that this configuration yields one of the lowest gaps, accross all experiments, of ~ 0.04. This could indicate slight underfitting, but overfitting stops being an issue here. Thus, in the future it would be beneficial to test lower α values, to see if we can improve the accu-



(a) Accuracy and error gap for varying mixing ratio values.

(b) Accuracy and error by weight penalty.

Figure 5. Accuracy and error by regularisation strength and misxing penalty respectively of combined L1 and L2 Regularisation.

racy and lower the errors. The best performance of the mixed experiments being with a lower α correlates with our L1 reguralization experiments. Keeping a relatively low L1 contribution ($\alpha = 0.25$) seems to be more effective than higher values, likely because excessive sparsity harms the model's ability to learn complex patterns in the data. Increasing the overall regularisation strength to $\lambda = 1 \times 10^{-3}$ does not yield better performance, as this configuration achieves lower validation accuracy and higher validation error, indicating potential underfitting. Thus, our question is answered: *the trade-off between L1-induced sparsity and L2-induced weight decay significantly affects the generalisation behaviour of the network. While purer L1 reguralization can increase sparsity, it tends to harm performance in our setting. A balanced or L2-dominant approach appears more effective for this task, reducing overfitting while maintaining strong predictive accuracy. This correlates with the nature of our dataset and model architecture, where weight decay helps control complexity without excessively discarding useful features.*

The highest accuracy accross all experiments is 85.3%, acheived by the soft-labelled baseline. However the training/validation errors are high, with a gap of ~ 0.500. The lowest training error accross all experiments is 0.241, acheived by the hard-labelled baseline model. However, this model has a validation error of 0.533 and a gap of ~ 0.370, indicating significant overfitting. The lowest validation error accross all experiments is 0.434, acheived by the Dropout-regularized model with an inclusion probability of 0.85. This model also achieves a high validation accuracy and maintains a small generalisation gap.

Thus, we select the best-performing configuration based on validation accuracy and generalisation behaviour: in our experiments this is the *Dropout-regularized model with an inclusion probability of* 0.85. When evaluated on the held-out test set, this model achieves a test accuracy of 85.1% and a validation error of 0.434, with a generalization gap of ~ 0.1. This reguralization technique effectively balances fitting the training data while maintaining strong performance on unseen data, demonstrating its effectiveness in improving generalisation in our neural network model.] .

4.1. Custom Activation Function

[Figure 6 presents the results of training with the proposed custom activation function

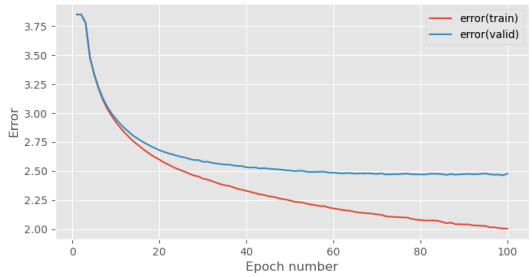
$$f(x) = \frac{1}{20 \cdot (1 + e^{-x})}, \tag{10}$$

where x is the input to the activation function.

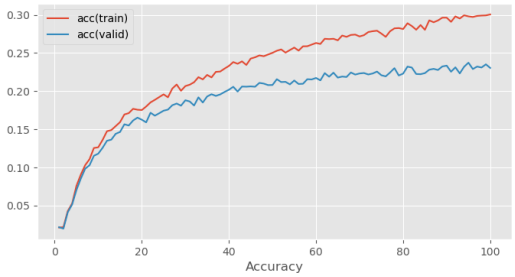
Figures 6a and 6b show the corresponding error and accuracy curves, while Figure 6c illustrates the gradient flow across layers.

The training and validation curves indicate that the model is *underfitting*. Training error decreases only slowly and does not reach the low values observed in our earlier experiments. The lowest error acheived is around ~ 2.000 at the last epoch. Validation accuracy, while increasing, is only around ~ 0.23, which is a very low result and significanly worse than past experiments. The gradient flow plot shows that the gradients in deeper layers rapidly shrink towards zero during training, highligting the fault of the network setup and design.

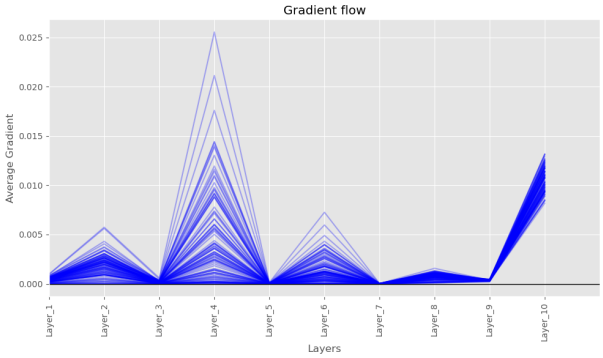
This behaviour is caused by a *vanishing gradient* prob-



(a) Training Error and Validation Error curves for the Custom Activation function experiment.



(b) Training Accuracy and Validation Accuracy curves for the Custom Activation function experiment.



(c) Gradient Flow Across Layers for the Custom Activation function experiment.

Figure 6. Figures depicting the Training Accuracy, Error, and Gradient Flow Across Layers for the 1 experiment run with the Custom Activation function

lem induced by the scaling factor $\frac{1}{20}$. The derivative of the activation is

$$f'(x) = \frac{1}{20}\sigma(x)(1-\sigma(x)),$$

where $\sigma(x)$ is the standard sigmoid[6]. For our networks, to minimize the loss we use gradient descent through back propagation. Thus, for example, the gradient of the loss at the first layer is given by

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdots \frac{\partial a_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1}$$

where a_i is the activation at layer i, n is the number of layers, and w_1 are the weights at the first layer. Each term $\frac{\partial a_i}{\partial a_{i-1}}$ includes the derivative of the activation function $f'(x)$. Since $f'(x)$ is scaled down by a factor of $\frac{1}{20}$ and the maximum value of the derivative occurs at $x = 0$, where $\sigma(0) = \frac{1}{2}$, we get:

$$f'(0) = \frac{1}{20} \cdot \frac{1}{2} \cdot (1 - \frac{1}{2}) = \frac{1}{80} = 0.0125 \qquad (11)$$

[6]Note that

$$\sigma(x) : \mathbb{R} \to [0,1], f(x) : \mathbb{R} \to \frac{1}{20} \cdot [0,1] = [0, 0.05]$$

Hence, the custom activation compresses outputs into a much smaller range compared to the standard sigmoid.

Thus, $\forall x \in \mathbb{R}$:

$$|f'(x)| \leq \max_x |f'(x)| = \frac{1}{80} = 0.0125$$

Multiplying such small derivatives across multiple layers leads to exponentially small gradients, especially in earlier layers. As a result, weights in those layers hardly change, the network fails to learn meaningful features, and both training and validation performance stagnate.

To address this issue, we can remove or substantially reduce the scaling (e.g. use the standard sigmoid or a milder scaling factor), or replace the custom activation with one that has better gradient propagation properties, such as ReLU or leaky ReLU (Goodfellow et al., 2016) (allowing for a minuscule gradient to flow in the negative interval). These activations have derivatives that do not saturate in the same way and would alleviate the vanishing gradient problem observed here. Furthermore, proper weight initialization paired with ReLU activations can help maintain healthy gradient flow during training. In 2013, Ester Hlav derived optimal initial variance of weight matrices in neural network layers with ReLU activation function (Hlav, 2023). In 2025, Alberto Fernández-Hernández et al. proposed a new deterministic (rather than random as we use in our experiments) approach to weight initialization (Fernández-Hernández et al., 2025). In summary, the custom activation's scaling causes vanishing gradients, leading to underfitting and poor performance. Adopting stan-

dard activations with better gradient properties would resolve these issues.] .

5. Conclusion

[In this coursework we systematically explored how network capacity and different regularisation techniques affect performance and overfitting on the EMNIST dataset. We varied the width and depth of a multilayer perceptron, studied label smoothing and dropout, and investigated L1, L2, and combined L1+L2 ("elastic-net") regularisation.

Our width and depth experiments confirmed that increasing capacity (more hidden units or more layers) reduces training error but eventually worsens generalisation: beyond a certain point, validation accuracy saturates while the generalisation gap grows. This behaviour matches the bias-variance trade-off described in the deep learning literature (e.g.(**Goodfellow et al., 2016**)), where larger models have higher with more *capacity* variance and are more prone to overfitting in the absence of sufficient regularisation.

In the context of our experiments, we can formalise *network capacity* via a capacity parameter $C \in \mathbb{N}$ that encodes the size of the network (**Goodfellow et al., 2016**) (e.g. total number of hidden units or layers). For each capacity level C, the network realises a hypothesis class $\mathcal{H}_C$, i.e. a set of functions $f_\theta : \mathcal{X} \to \mathcal{Y}$ parameterised by $\theta \in \Theta_C$, with $\mathcal{H}_C \subseteq \mathcal{H}_{C'}$ whenever $C < C'$ and $|\Theta_C| < |\Theta_{C'}|$. Increasing either the width or the depth of the MLP corresponds to increasing C, thereby enlarging $\mathcal{H}_C$ and the set of functions the model can approximate. Let $\mathcal{L}_{\text{CE}}$ denote the cross-entropy loss and define the empirical training and validation risks for a given capacity C as

$$\hat{R}_{\text{train}}(\theta; C) = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \mathcal{L}_{\text{CE}}$$

$$\hat{R}_{\text{val}}(\theta; C) = \frac{1}{n_{\text{val}}} \sum_{j=1}^{n_{\text{val}}} \mathcal{L}_{\text{CE'}}.$$

Where $\mathcal{L}_{\text{CE'}}$ is the cross-entropy validation loss. Empirically, when we optimise θ for each C, we observe that $\hat{R}_{\text{train}}(\theta_C^\star; C)$ is approximately a *monotonically decreasing* function of C: wider and deeper networks fit the training data more easily. In contrast, the validation risk $\hat{R}_{\text{val}}(\theta_C^\star; C)$ exhibits a U-shaped behaviour in C: it decreases from a high value at small capacities (underfitting), reaches a minimum at some intermediate $C^\star$, and then increases again for very large capacities (overfitting). This behaviour can be summarised by the generalisation gap

$$g(C) = \hat{R}_{\text{val}}(\theta_C^\star; C) - \hat{R}_{\text{train}}(\theta_C^\star; C).$$

For very small C, both $\hat{R}_{\text{train}}$ and $\hat{R}_{\text{val}}$ are large and $g(C)$ is small (high bias, low variance). As C increases towards $C^\star$, both quantities decrease and $g(C)$ remains moderate. Beyond $C^\star$, $\hat{R}_{\text{train}}$ continues to decrease while $\hat{R}_{\text{val}}$ stagnates or increases, so $g(C)$ grows: the model moves from an underfitting regime to an overfitting regime in which extremely low training error is achieved at the cost of worse validation performance. Our width and depth sweeps provide an explicit empirical instance of this phenomenon, with the capacity parameter C realised concretely via the number of hidden units and layers in the network.

Among the regularisation methods, both dropout and L2 penalties (and their combination) improved generalisation when appropriately tuned. Dropout alone reduced overfitting by randomly disabling units during training, leading to slightly higher validation accuracy and a smaller generalisation gap than the baseline in our best configuration. L2 weight decay and the mixed L1+L2 penalty further controlled the magnitude of the weights; in some settings these methods matched or modestly exceeded our baseline in terms of validation performance, while also producing smoother error curves. However, future tasks include performing experiments to perhaps outperform the Dropout-methods for certain cases. Overall, dropout tends to be more effective at preventing co-adaptation of units, whereas L1/L2 regularisation primarily constrains the complexity of the learned parameters.

Beyond these techniques, the literature offers many additional strategies for combating overfitting. For instance, data augmentation has been widely used to improve generalisation by synthetically increasing dataset diversity (e.g. **Oh & Yun (2024)**). There are many advanced architectures and training techniques that could be explored to improve performance on the EMNIST dataset and similar tasks. Today, common methods include *CutOut and CutMix* (**Oh & Yun, 2024**), which have been proven to improve generalisation in image classification tasks. In 2022, *MixupE* (**Zou et al., 2023**) was proposed, which is an improvement of a method called *Mixup* (CutMix is an extension of Mixup), which is a popular data augmentatiopn technique. Here, they performed experiments where they use a variety of large-scale image classification datasets, including ImageNet. Incorporating such methods into our setup would be a natural extension of this work. In Appendix **5**, I attempt to implement Mixup and CutMix data augmentation techniques, but was unable to complete the experiments in time for submission. However, I include the implementation details and preliminary results in the appendix for reference. While my results are inconclusive, they provide a foundation for future exploration of these techniques on EMNIST, the next thing I would attempt is to establish a framwork to control the network capacity in various ways, since I believe the main bottleneck is caused by the capacity of our model setup and lack of time for optimization.

For future work, I would like to *(i)* explore richer data-augmentation schemes tailored to EMNIST (e.g. elastic distortions and random affine transformations

(**Simard et al., 2003**)), *(ii)* **investigate modern optimisers and learning-rate schedules (such as cosine annealing (Loshchilov & Hutter, 2017)) that may further improve training stability, and** *(iii)* **study more advanced architectures with carefully controlled capacity, such as residual connections (He et al., 2015) which offer a way to increase depth while mitigating optimisation issues, and would allow us to study how skip-connection-based architectures interact with explicit regularisation on EMNIST. or convolutional layers, to understand how architectural inductive biases interact with regularisation. These directions would deepen our understanding of how network capacity and regularisation jointly determine the trade-off between fitting the training data and achieving good generalisation.]** .

[

Appendix: Mixup and CutMix on EMNIST

Setup. 3-layer MLP (ReLU, hidden width 128), Softmax output, Adam optimiser. Initial runs at learning rate 10^{-2} diverged; the follow-up sweep used learning rate 3×10^{-4}, batch size 256, 100 epochs. Data augmentation used Mixup and CutMix via `MixupCutmixDataProvider` (Beta-α sampling with permutation and CutMix area correction); loss was CrossEntropy with soft labels.

Configurations.

Run	Train err.	Val. err.	Val. Acc.	Runtime (s)
baseline_lr3e-4	3.320	3.363	0.523	289
mixup_a0.4	3.364	3.280	0.607	307
cutmix_a0.8	3.484	3.309	0.577	305
mix+cut_combo	3.425	3.302	0.585	310

Table 5. Mixup/CutMix ablations (learning rate 3×10^{-4}, batch size 256, 100 epochs).

Outcome. All runs remained near-chance performance (chance $\approx$ 2% for 47 classes): training error $\sim$ 3.3 (vs. 3.85 for uniform softmax) and validation accuracy $\leq$ 0.61. The augmentation code includes (Beta-α sampling, permutation, CutMix area-adjusted labels); the bottleneck is optimisation and capacity: the MLP barely learns before augmentation is applied.

Fixes proposed for follow-up. (i) Lower learning rate to 1×10^{-4} (or 7×10^{-5} if still unstable); (ii) increase width to 256 and optionally insert `DropoutLayer` (incl_prob 0.8) after each ReLU for stable training; (iii) enable label smoothing on train only (`smooth_labels=True`) to stabilise soft targets; (iv) reduce augmentation strength: Mixup $\alpha = 0.8$, prob 0.7; CutMix $\alpha = 1.0$, prob 0.5; Combo Mixup $\alpha = 0.4$ + CutMix $\alpha = 0.8$, probs 0.5/0.3. Expectation: recover ~0.8+ validation accuracy baseline and allow Mixup/CutMix to add modest gains once the optimiser is stable.]

References

Fernández-Hernández, Alberto, Mestre, Jose I., Dolz, Manuel F., Duato, Jose, and Quintana-Ortí, Enrique S. Sinusoidal initialization, time for a new start, 2025. URL https://arxiv.org/abs/2505.12909.

Glorot, Xavier and Bengio, Yoshua. Understanding the difficulty of training deep feedforward neural networks. In Teh, Yee Whye and Titterington, Mike (eds.), *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pp. 249–256, Chia Laguna Resort, Sardinia, Italy, 13–15 May 2010. PMLR. URL https://proceedings.mlr.press/v9/glorot10a.html.

Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

He, Kaiming, Zhang, Xiangyu, Ren, Shaoqing, and Sun, Jian. Deep residual learning for image recognition, 2015. URL https://arxiv.org/abs/1512.03385.

Hlav, Ester. Kaiming he initialization in neural networks – math proof, feb 2023. URL https://towardsdatascience.com/kaiming-he-initialization-in-neural-networks-math-proof-73b9a0d845c Published on Towards Data Science.

Loshchilov, Ilya and Hutter, Frank. Sgdr: Stochastic gradient descent with warm restarts, 2017. URL https://arxiv.org/abs/1608.03983.

Ng, Andrew Y. Feature selection, l1 vs. l2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 78, 2004.

Oh, Junsoo and Yun, Chulhee. Provable benefit of cutout and cutmix for feature learning, 2024. URL https://arxiv.org/abs/2410.23672.

Simard, P.Y., Steinkraus, D., and Platt, J.C. Best practices for convolutional neural networks applied to visual document analysis. In *Seventh International Conference on Document Analysis and Recognition, 2003. Proceedings.*, pp. 958–963, 2003. doi: 10.1109/ICDAR.2003.1227801.

Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1): 1929–1958, 2014.

Zou, Hui and Hastie, Trevor. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 67(2): 301–320, 03 2005. ISSN 1369-7412. doi: 10.1111/j.1467-9868.2005.00503.x. URL https://doi.org/10.1111/j.1467-9868.2005.00503.x.

Zou, Yingtian, Verma, Vikas, Mittal, Sarthak, Tang, Wai Hoh, Pham, Hieu, Kannala, Juho, Bengio, Yoshua, Solin, Arno, and Kawaguchi, Kenji. Mixupe: Understanding and improving mixup from directional derivative perspective, 2023. URL https://arxiv.org/abs/2212.13381.